



UNIVERSIDADE VEIGA DE ALMEIDA

Bacharelado em Ciência da Computação

GABRIEL ALBUQUERQUE VARELA SANTARELLO - 1240110815

CAUÃ MANUEL PROENÇA DE ANDRADE - 1240109764

IGOR ROCHA LOBATO - 1240114118

CAIO PARADA OLIVEIRA PLANINSCHKEK – 1240205596

JOÃO VICTOR BERÇOT CHABUDET CABRAL – 1240108001

PRESENTE!

**DISPOSITIVO EMBARCADO PARA CONTABILIZAÇÃO DE PRESENÇA EM SALA
DE AULA**

(Trabalho da disciplina de Sistemas Embarcados)

Entrega 05 – Protótipo eletrônico com testes documentados

Professor: Thiago Alberto Ramos Gabriel

Rio de Janeiro RJ

2026

1 DESCRIÇÃO DA INTEGRAÇÃO

O sistema registra a presença do aluno em sala pelo crachá, sem chamada oral e sem lista em papel. O protótipo se organiza em três blocos, na ordem em que a informação atravessa o aparelho.

O sensor é o leitor RFID/NFC RC522, de 13,56 MHz. Ele não lê nome nem matrícula: lê o **UID** da tag, o número único que cada crachá carrega de fábrica, com 4 bytes nas tags Mifare e 7 bytes nas NTAG215. A comunicação com o microcontrolador é feita por **SPI**, no barramento VSPI por hardware, com o sinal de seleção no GPIO 5, o relógio no 18, os dados de ida no 23, os de volta no 19 e o reinício no 22. O módulo trabalha estritamente em 3,3 V, tensão que o regulador da própria placa fornece.

O processamento é o ESP32 DevKit V1. Ele recebe o UID e decide sem consultar rede nenhuma: se aquele crachá ainda não foi registrado nesta sessão, o evento é gravado; se o mesmo crachá voltar ao campo em menos de cinco segundos, nada acontece, porque é a mesma aproximação sendo lida de novo; e se ele já estava registrado, o aparelho avisa que reconheceu, mas não grava uma segunda vez. A decisão se apoia numa lista gravada no sistema de arquivos interno do próprio microcontrolador, o LittleFS, relida toda vez que o aparelho liga. É o que impede que uma queda de energia no meio da aula apague a chamada.

Os atuadores são o LED RGB, ligado aos GPIO 25, 26 e 27, **e o buzzer**, comandado pelo GPIO 33 por meio de um transistor TIP122. O pino do ESP32 entrega 3,3 V e pouca corrente, e o transistor permite alimentar o buzzer com os 5 V da placa. Eles carregam a resposta que o aluno vê e ouve, e as três respostas são deliberadamente diferentes entre si: duas piscadas verdes com um bip de 100 ms quando a presença entra; uma piscada verde, sem som, quando o crachá já estava registrado; e nada, nem luz nem som, na releitura dentro da janela de cinco segundos. O aluno não precisa olhar para tela nenhuma para saber o que aconteceu.

O ciclo completo, portanto, é **crachá → RC522 → ESP32 → decisão → LED e buzzer → registro em disco**, e se fecha inteiro dentro do aparelho, sem internet. A rede serve a dois momentos que não são esse: o professor abre a sessão pelo celular, numa página servida pelo próprio ESP32, e o relatório sobe para a nuvem no fim da aula.

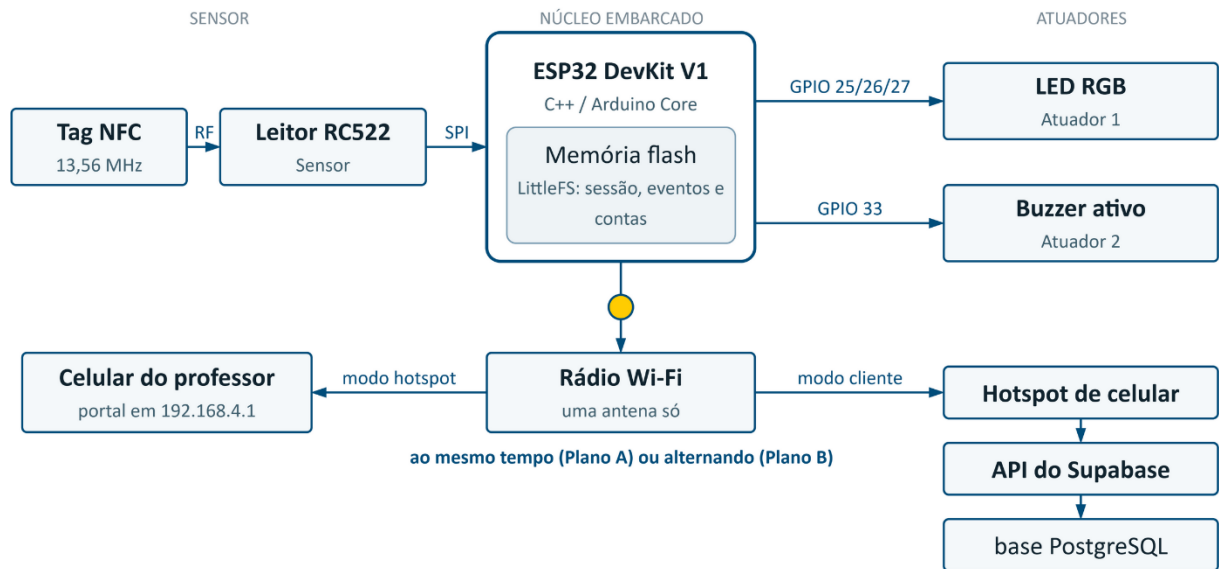


Figura 1 – Diagrama de blocos do sistema.

1.1 Ligações do ESP32

Componente	Pino do módulo	GPIO	Direção
RC522 (sensor)	SDA / SS	5	Saída (SPI CS)
RC522	SCK	18	Saída
RC522	MOSI	23	Saída
RC522	MISO	19	Entrada
RC522	RST	22	Saída
LED RGB (atuador 1)	R	25	Saída
LED RGB	G	26	Saída
LED RGB	B	27	Saída
Buzzer (atuador 2)	sinal	33	Saída

Uma fonte de 5 V na tomada alimenta o conjunto, porque o aparelho é fixo e foi pensado para ficar preso à parede da sala; o leitor recebe os 3,3 V do regulador da própria placa. Cada linha do LED tem um resistor de 300 Ω em série, e o buzzer é acionado pelo GPIO 33 através do transistor, com o positivo ligado aos 5 V. O projeto usa **um sensor só**: o professor dispensou esta equipe da exigência de dois.

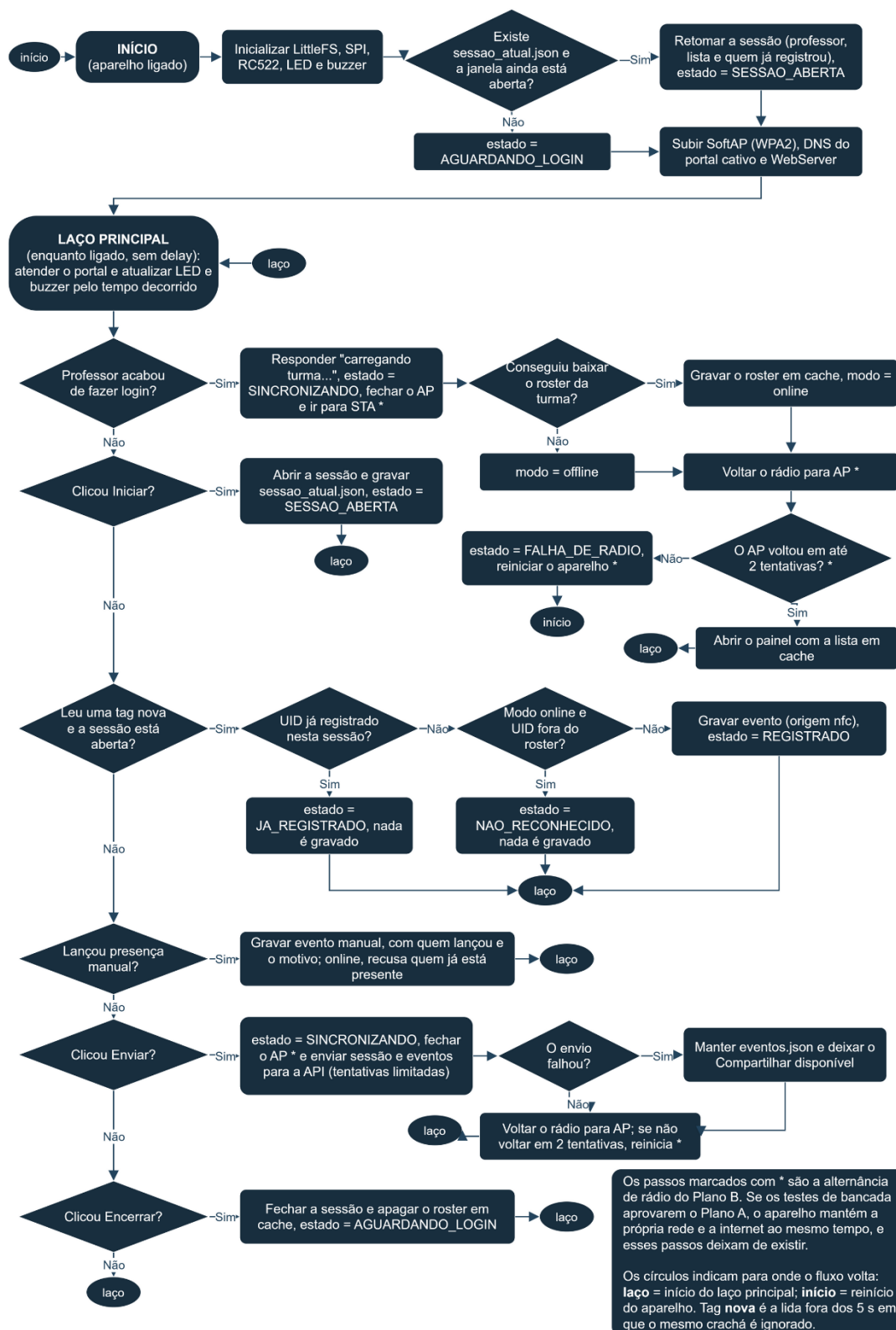


Figura 2 – Fluxograma da lógica do programa.

2 CÓDIGO-FONTE ATUALIZADO

O código está no repositório público do projeto, em github.com/caioplaninschek/presente. A Entrega 04 foi entregue sem nenhuma linha de

firmware escrita para as peças; desde então o repositório recebeu **seis programas, três módulos compartilhados, o portal do professor e o banco na nuvem**.

A decisão de organização mais importante do período foi escrever **um programa por experimento**. Em vez de um firmware único e grande, cada coisa que precisa ser provada na placa virou um programa próprio, na sua pasta em `src/`, com o seu ambiente de compilação declarado no `platformio.ini`. A razão é que há um microcontrolador só para cinco pessoas: com os programas separados, cada frente escreve o seu sem esbarrar na do outro, e quem está com a placa grava um de cada vez, sabendo exatamente o que aquele teste prova e o que ele não prova.

Pasta em <code>src/</code>	O que o programa faz	Testes que ele atende
<code>ambiente/</code>	Pisca o LED da placa e escreve uma frase por segundo no monitor serial. Prova que o computador compila e grava	Nenhum. É pré-requisito de todos
<code>prova-de-vida/</code>	Percorre vermelho, verde e azul, um segundo cada, e toca o bip. O nome da cor acesa sai no monitor serial	Atuadores isolados
<code>leitor/</code>	Encosta a tag, o UID aparece no monitor serial em hexadecimal maiúsculo. Sem LED, sem som, sem gravar	Sensor isolado
<code>encostou/</code>	Junta os dois: encostou, piscou verde, apitou, e não registra duas vezes	Testes 1 e 2
<code>persistencia/</code>	O mesmo, com o arquivo de eventos atrás: a lista de quem já está registrado sobrevive ao reinício	Teste 8
<code>radio/</code>	Mede a memória livre no instante da conexão segura e observa o comportamento do painel no pulo de canal	Testes 7c e 7d

O programa gravado no protótipo, que rodou nos testes 1 e 2 e é o mesmo apresentado na demonstração em laboratório, é o `encostou`, no arquivo `src/encostou/main.cpp`. O arquivo abre com um comentário que explica o que o programa faz, a que testes atende e em que módulo da pasta `lib/` está cada parte do mecanismo; os trechos principais, como a decisão entre registrar e recusar e a temporização das piscadas e do bip sem espera bloqueante, têm comentário próprio.

Para que esses programas não repetissem o mesmo código, o que é comum a eles vive em três módulos, na pasta `lib/`. O módulo `rfid/` faz a leitura do UID de 4 e de 7 bytes e controla a janela de silêncio de cinco segundos; o `feedback/` comanda o LED e o buzzer por máquina de estados, sem nunca travar o programa numa espera; e o `storage/` grava os eventos no LittleFS, uma linha por evento, em modo de acréscimo, relendo o arquivo quando o aparelho liga.

As versões das bibliotecas estão fixadas de propósito no `platformio.ini`: a plataforma `espressif32` na 7.1.3, a `MFRC522` na 1.4.12 e a `ArduinoJson` na 6.21.6. O firmware precisa compilar igual nas cinco máquinas da equipe, e mudar uma versão é decisão de grupo, não de quem estiver editando no momento. O sistema de arquivos também está declarado

explicitamente como LittleFS, em lugar do padrão da plataforma, para que a gravação do sistema de arquivos não apague o portal já gravado na mesma partição.

Quatro regras de escrita valem para todos os programas e vêm da especificação: nenhuma espera bloqueante no laço principal, com todo tempo contado por marcos de relógio; SPI por hardware; documento JSON de tamanho fixo, sem concatenar texto em memória; e log no monitor serial com níveis, porque é dele que saem os números dos testes.

Fora do firmware, o mesmo período produziu as três telas do professor (login, painel e relatório), que rodam no navegador contra dados de mentira, e o banco na nuvem no Supabase, com o esquema, os dados de teste e um roteiro de verificação. Nenhum dos dois está dentro do aparelho ainda; os dois pertencem à etapa de integração.

A especificação do projeto acompanhou o código. Ela passou por quatro rodadas de decisão entre 13 e 16 de setembro, e três delas alcançam diretamente este protótipo: uma presença por crachá por sessão, decidida pela lista em disco e não pela memória; os 200 ms do teste 1 contados do início da conversa com o leitor, e declarados como piso; e o formato do que o aparelho grava em disco, que passou a ser uma linha por evento, com o tempo medido desde o instante em que a placa ligou.

3 TESTES DOCUMENTADOS

Os testes são executados na placa real, com o aparelho montado, e verificam apenas o comportamento externo: o que o LED e o buzzer fazem, o que sai no monitor serial e o que fica gravado em disco. Cada teste deixa evidência gravada, que pode ser uma foto, um vídeo curto ou o trecho do log com o horário, e ela fica versionada no repositório.

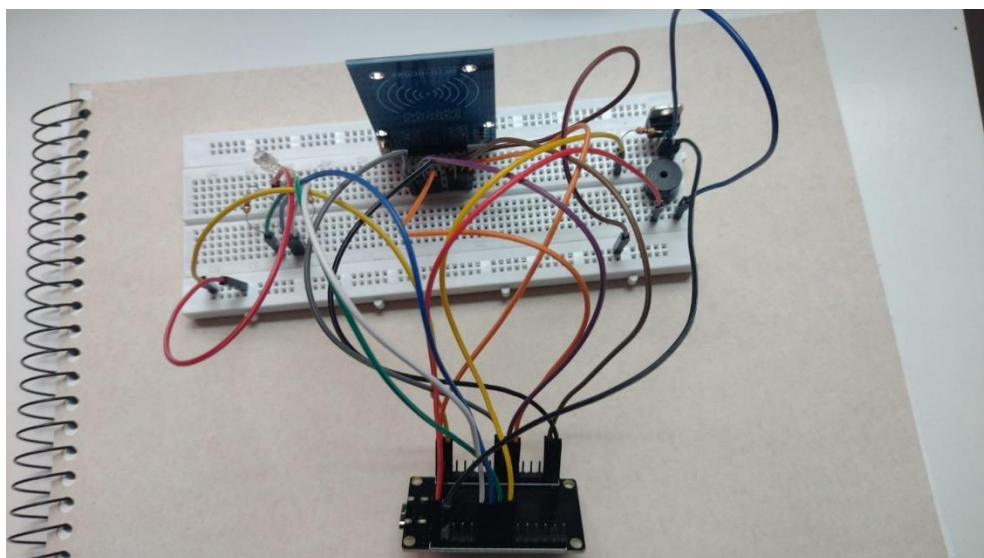


Figura 3 – Protótipo montado: o ESP32 embaixo e, na protoboard, o LED, o leitor, o transistor e o buzzer.

A Figura 3 mostra o protótipo usado nos testes. O leitor RC522, o LED RGB, o transistor e o buzzer estão montados numa protoboard de 16,6 cm por 5,5 cm, com 1,0 cm de altura, e o ESP32 fica fora dela, ligado por jumpers.

3.1 Prova de vida: as três cores do LED e o bip

Não é um dos testes do plano de testes da especificação: é a checagem dos atuadores isolados, feita antes do leitor. Entra aqui porque rodou e revelou o problema do buzzer.

Descrição: com o programa de prova de vida, o LED percorre vermelho, verde e azul, um segundo cada, apaga e toca um bip de 100 ms, e o monitor serial imprime o nome da cor acesa.

Resultado esperado: as três cores acendem uma de cada vez, o nome no monitor serial bate com a cor acesa, e o bip é audível a um passo da placa.

Resultado obtido: as três cores acenderam na ordem e bateram com o monitor serial. O bip saiu muito baixo e com um som estranho, audível só de perto.

Problemas e correções: o buzzer é de 5 V, e a ligação foi feita com um transistor TIP122, como a especificação previa para esse caso. Mesmo assim, o que se ouve é um clique curto, parecido com o de um botão de mouse, e não um bip. O João testou a peça à parte, ligada a um Arduino com um programa de teste, e ela também não apitou; a hipótese dele é que o disco cerâmico do buzzer esteja danificado. A peça vai ser trocada.

Evidência: vídeo das cores, gravado pelo João em 21/09 (docs/assets/testes/prova-de-vida-2026-09-21.mp4), e o trecho do monitor serial colado na issue da prova de vida (#24).

3.2 Teste 1: Crachá de 4 bytes, três aproximações seguidas

Descrição: encostar uma tag Mifare, de UID com 4 bytes, três vezes em sequência.

Resultado esperado: um único registro, e não três, com a resposta do LED e do buzzer em menos de 200 ms, contados do início da conversa com o leitor. O número é declarado como piso, e não como medida exata: o tempo em que a tag espera no campo até o programa ir buscá-la não é visível de dentro do aparelho.

Resultado obtido: com o cartão Mifare (UID 62EF7E05), o primeiro toque registrou a presença, com duas piscadas verdes. Alguns segundos depois, o mesmo cartão foi encostado de novo e recebeu uma piscada verde, com o veredito "já registrado" no monitor serial e nenhum registro novo. Encostado logo em seguida, ainda dentro dos cinco segundos de silêncio, foi ignorado: o LED não acendeu e nada saiu no monitor. Passado esse intervalo, um novo toque deu outra vez uma piscada e nenhum registro. O chaveiro (UID 51A79F97), encostado na sequência, registrou normalmente, com duas piscadas, porque cada crachá tem o seu próprio

registro. Nas cinco leituras do log, o tempo entre o início da conversa com o leitor e o começo do feedback ficou entre 29,2 e 29,3 ms, abaixo do piso de 200 ms, e a decisão levou no máximo 0,1 ms. O teste passou no registro, na janela de silêncio e no tempo de resposta; a parte sonora não passou, porque o buzzer não apita.

Problemas e correções: no primeiro registro de cada crachá, quando o programa aciona o buzzer por 100 ms, ouviu-se apenas o clique curto já observado na prova de vida, sem bip. O clique também aparece quando o LED acende na recusa do crachá já registrado, momento em que o programa mantém o buzzer desligado, e não aparece no toque dentro dos cinco segundos, em que o LED fica apagado. Portanto, o clique acompanha o acendimento do LED, mesmo quando o programa não aciona o buzzer. As hipóteses são duas: o clique viria de uma interferência do acionamento do LED no circuito do buzzer, e a falta do bip, da peça danificada. Nenhuma das duas foi confirmada até esta entrega. O registro, a recusa e a janela de silêncio funcionaram na primeira gravação, sem correção no programa.

Evidência: vídeo gravado pelo João às 2h35 de 22/09 (docs/assets/testes/encostou-2026-09-22.mp4) e o log do monitor serial, colado na issue do sinal de presença (#26):

```
62EF7E05 4 bytes registrado decidir: 0.1 ms toque->feedback: 29.3 ms
62EF7E05 4 bytes ja registrado decidir: 0.0 ms toque->feedback: 29.2 ms
62EF7E05 4 bytes ja registrado decidir: 0.0 ms toque->feedback: 29.2 ms
51A79F97 4 bytes registrado decidir: 0.0 ms toque->feedback: 29.2 ms
51A79F97 4 bytes ja registrado decidir: 0.0 ms toque->feedback: 29.2 ms
```

3.3 Teste 2: Crachá de 7 bytes

Descrição: encostar uma tag NTAG215, de UID com 7 bytes.

Resultado esperado: o registro acontece pelo UID, e nada é escrito na tag.

Resultado obtido: no programa que apenas lê o crachá, a tag adesiva foi lida com UID de 7 bytes (04B5A979C32A81), ao lado do cartão e do chaveiro, ambos de 4 bytes, e cada crachá devolveu sempre o mesmo número. Nos programas do sinal de presença e da persistência, só o cartão e o chaveiro foram usados, e o registro com a tag de 7 bytes não foi obtido até o envio.

Problemas e correções: a leitura exigiu aproximação cuidadosa, sobretudo com a tag adesiva: a antena dela é pequena, e o leitor só a detecta bem encostada. Nenhuma correção foi feita até esta entrega; a equipe avalia trocar a tag adesiva por uma NTAG215 em formato de cartão, de antena maior. Quanto à escrita, o programa não contém nenhum comando que grave na tag, de modo que a garantia de que nada é escrito no crachá vem do código, e não de uma observação.

Evidência: a lista dos UIDs lidos, colada na issue do leitor (#25):

04B5A979C32A81	7 bytes	tag adesiva
51A79F97	4 bytes	chaveiro
62EF7E05	4 bytes	cartão

3.4 Teste 8: Estresse: 30 toques com um reinício no meio

Descrição: trinta aproximações seguidas, com o aparelho desligado e religado no meio da sequência.

Resultado esperado: nenhuma duplicata, ou seja, um registro por crachá, com quem já estava registrado continuando recusado depois do reinício; e nenhuma perda do que já havia sido gravado.

Resultado obtido: obtido em parte. O teste rodou com dois crachás, o cartão (UID 62EF7E05) e o chaveiro (UID 51A79F97), em vez dos trinta toques previstos. O cartão foi registrado primeiro; em seguida, a alimentação da placa foi cortada e religada, e o chaveiro foi registrado depois da volta. O próprio arquivo de eventos confirma o reinício: cada linha guarda o tempo contado desde que a placa ligou, e a do chaveiro, gravada depois, marca 9,3 s, contra 201,9 s na do cartão. Os dois crachás foram então encostados de novo, e o programa respondeu "já registrado" a ambos, sem gravar nada. A contagem que acompanha as duas respostas, de dois crachás no total, mostra que elas vieram depois do registro do chaveiro e, portanto, depois do reinício. Como o registro do cartão é anterior ao corte de energia, a recusa dele prova que a lista de quem já estava registrado foi relida do arquivo quando a placa voltou. O arquivo terminou com duas linhas, uma por crachá, sem duplicata e sem perda de registro.

Problemas e correções: a rodada foi menor que a prevista, com dois crachás, e nem o número de toques nem o momento exato do corte de energia foram anotados. Só o cartão e o chaveiro são detectados pelo leitor com folga, porque a tag adesiva tem antena pequena, como descrito no teste 2. Também não foi guardada a linha que o programa imprime ao ligar, com o total relido do arquivo; a releitura fica demonstrada pela recusa do cartão. O programa não precisou de correção. A rodada completa, com trinta toques e mais crachás, fica para a etapa de integração.

Evidência: o conteúdo do arquivo de eventos, impresso pelo comando "l" do programa, e as respostas do monitor serial, colados na issue da persistência (#27):

```
--- eventos.json ---
{"uid":"62EF7E05","ms_desde_boot":201857,"origem":"nfc"}
{"uid":"51A79F97","ms_desde_boot":9304,"origem":"nfc"}
--- 2 linha(s); 2 cracha(s) diferente(s) registrado(s) ---
62EF7E05 4 bytes ja registrado - nada gravado (2 no total)
51A79F97 4 bytes ja registrado - nada gravado (2 no total)
```

3.5 Teste 7d: Memória livre no instante da conexão segura

Descrição: com a rede do aparelho, o serviço de nomes, o servidor de páginas e a lista da turma de pé, medir a memória livre no instante em que a conexão segura com a nuvem é aberta.

Resultado esperado: pelo menos 40 KB livres. Abaixo disso, o arranjo simultâneo reprova. Medido sem nenhum celular conectado ao aparelho o número sai otimista, e um valor entre 40 e 45 KB é suspeito, não aprovado.

Resultado obtido: 236,5 KB livres imediatamente antes da conexão segura, sem nenhum celular conectado ao aparelho, o que cai na faixa de aprovação (acima de 45 KB). Durante a conexão sobraram 196,3 KB, e o menor valor desde a inicialização foi 186,6 KB. Foi registrada uma rodada, e não as três previstas.

Problemas e correções: três, nenhum de memória. O endereço do banco não respondia, porque o projeto gratuito na nuvem tinha sido pausado por falta de uso, e o Caio o reativou. O monitor mostrou o aviso request handler not found enquanto havia aparelho conectado à rede da placa; o aviso é inofensivo, e a medição foi refeita sem ninguém nessa rede. Numa tentativa, a conexão falhou por não encontrar o endereço do banco na internet do celular; repetida depois de reiniciar a placa, passou.

Evidência: trecho do monitor serial colado na issue do teste de memória (#17).

3.6 Teste 7c: O painel sobrevive ao pulo de canal

Descrição: com o professor logado no painel pelo celular, o aparelho conecta a um hotspot que está em outro canal.

Resultado esperado: o painel continua aberto, sem pedir login de novo; se cair, reconecta sozinho em menos de cinco segundos. Com dois celulares do mesmo sistema operacional, o veredito sai parcial.

Resultado obtido: não obtido. Na única rodada registrada, o aparelho já estava conectado ao hotspot quando o botão foi apertado no celular, e o monitor avisou que a medição não teria pulo de canal. A página continuou aberta, mas sem passar pela troca de canal, que é o que o teste mede. Com um celular conectado, a memória livre antes da conexão segura foi de 229,6 KB, o que confirma o teste 7d.

Problemas e correções: a rodada precisa ser refeita desde a reinicialização: reiniciar a placa, conectar o celular do professor, fazer login e só então pedir a conexão ao hotspot. A escolha entre manter as duas redes ao mesmo tempo e alterná-las espera essa rodada válida.

Evidência: o trecho do monitor serial da rodada, com o aviso de que ela não teria pulo de canal, colado na issue do teste do painel (#18).

4 O QUE NÃO ENTRA NESTA ENTREGA, E POR QUÊ

O plano de testes da especificação tem onze itens, e esta entrega apresenta cinco. A ausência dos outros é escolha declarada, não descuido, e cada grupo tem um motivo próprio.

Os testes 3, 4, 5 e 6 (crachá fora da turma, modo offline, reenvio do relatório e dois professores em sequência) dependem de a lista da turma estar dentro do aparelho, e ela só passa a existir quando o ESP32 falar com a nuvem, na etapa de integração prevista entre o fim de setembro e o começo de outubro. Sem essa lista, o aparelho não tem como saber que um crachá não pertence à turma, e os quatro testes não teriam o que verificar.

Os testes 7 e 7b só se tornam obrigatórios se a medição do rádio indicar o arranjo alternado. Eles verificam a alternância entre a rede do aparelho e a rede externa, que deixa de existir caso o arranjo simultâneo seja aprovado. A decisão sai dos testes 7c e 7d desta entrega.

A máquina de estados completa do firmware integrado, com os estados de rede e de sessão, e a retomada da sessão depois de um reinício também ficam para a integração, porque dependem do portal rodando dentro do aparelho, e o portal ainda roda no navegador.